

DVSA Project Report

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section: ICS-344

Student Name(s) + ID(s): Omar AlJughaiman **202172810** , **Team28** , **Sec04**

University: King Fahd University of Petroleum and Minerals (KFUPM)

AWS Region: us-east-1

Your DVSA Website URL: <http://dvsa-omar-352712712043-us-east-1.s3-website.us-east-1.amazonaws.com/store>

Date: 2026-05-05

Vulnerability Title: Lesson #1: Event Injection

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an event injection vulnerability in DVSA that leads to remote code execution (RCE)[cite: 5]. The issue exists in the `/order` API, where user input is processed

unsafely[cite: 5]. As a result, an attacker can inject and execute arbitrary JavaScript code inside the Lambda function[cite: 5]. This allows access to the backend environment, including file operations and system resources[cite: 5].

Part 2) Why This Works / Root Cause

The vulnerability is caused by unsafe deserialization using the `node-serialize` library[cite: 5]. This library allows serialized functions to be executed when parsing input[cite: 5]. Since the application directly unserializes user-controlled data, an attacker can inject JavaScript code that runs immediately during request processing[cite: 5].

Part 3) Environment and Setup

- **Target Endpoint:** The `/dvsa/order` API endpoint through API Gateway[cite: 5].
- **AWS Service:** AWS Lambda function (`DVSA-ORDER-MANAGER`)[cite: 5].
- **Tools Used:** Terminal/Git Bash (`curl` command), AWS Console (CloudWatch Logs)[cite: 5].

Part 4) Reproduction Steps

1. Identify the `/dvsa/order` API endpoint from API Gateway[cite: 5].
2. Prepare a malicious payload containing a serialized function using the `__$ND_FUNC__$` marker[cite: 5].
3. The payload creates a file in `/tmp` , reads it, and logs its content[cite: 5].
4. Send the payload using a `POST` request to the `/order` endpoint[cite: 5].
5. Observe that the API returns an error[cite: 5].
6. Check CloudWatch logs to verify whether the injected code executed[cite: 5].

Part 5) Evidence and Proof

The API response returned an internal server error; however, this does not indicate failure[cite: 5]. The CloudWatch logs show the message "FILE READ SUCCESS," which confirms that the injected code executed successfully[cite: 5]. The log also includes the content written to the file, proving that the attacker was able to create and read a file inside the Lambda environment[cite: 5]. This demonstrates successful remote code execution[cite: 5].

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Logs

Log Management

Log Analytics

Log Anomalies

Display

Timestamp	Message
2026-05-02T21:03:02.379Z	2026-05-02T21:03:02.379Z 44c5c168-9604-4223-823a-2c4b6b030b79 ERROR FILE READ SUCCESS: You ...
2026-05-02T21:03:02.379Z	44c5c168-9604-4223-823a-2c4b6b030b79 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
2026-05-02T21:03:02.380Z	2026-05-02T21:03:02.380Z 44c5c168-9604-4223-823a-2c4b6b030b79 ERROR Invoke Error {"errorTyp...

aws

Search

[Alt+S]

United States (N. Virginia)

omaraju1 (352712712043)

CloudWatch

Log management

/aws/lambda/DVSA-ORDER-MANAGER

2026/05/03/[\$LATEST]f39ec101a95c4ca2b3faf5b...

CloudWatch

Favorites and recents

(APM)

Infrastructure Monitoring

Logs

Log Management

Log Analytics

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

Log events

file read

1m

1h

UTC timezone

Display

Timestamp	Message
2026-05-03T05:52:16.857Z	2026-05-03T05:52:16.857Z d7ed88da-a610-4d6e-8612-25426793a79b ERROR Invoke Error {"errorTyp...
2026-05-03T05:54:50.283Z	2026-05-03T05:54:50.283Z 8b845786-666b-4ba2-a1fb-7429225471c9 ERROR Invoke Error {"errorTyp...

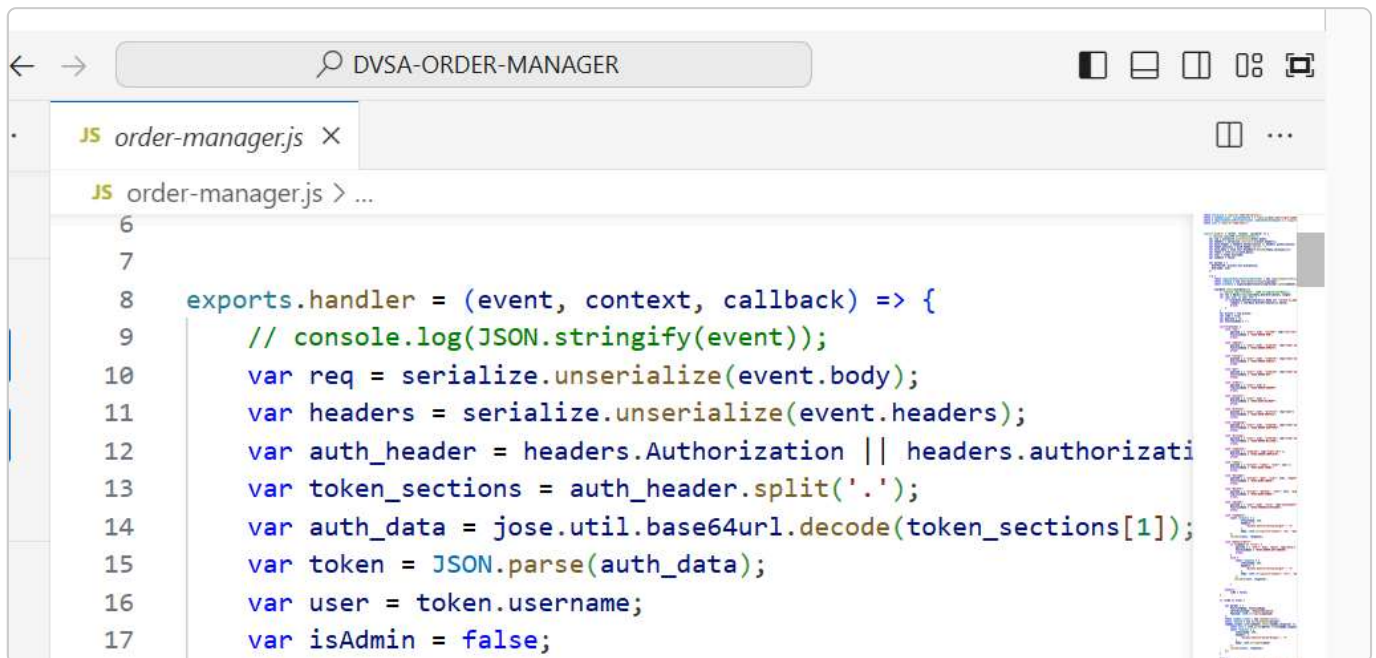
Part 6) Fix Strategy / Probable Mitigation

The fix should be applied in the DVSA-ORDER-MANAGER Lambda function[cite: 5]. The unsafe deserialization using `node-serialize` must be removed and replaced with safe JSON parsing[cite: 5]. This prevents the application from executing attacker-controlled input and ensures that requests are treated as data only[cite: 5].

Part 7) Code / Config Changes

The fix was applied in the DVSA-ORDER-MANAGER Lambda function by replacing `serialize.unserialize()` with `JSON.parse()` [cite: 5]. Additionally, headers are now accessed directly using `event.headers` instead of being unserialized[cite: 5]. This removes the ability to execute injected code from user input[cite: 5].

```
// BEFORE (Vulnerable)
var req = serialize.unserialize(event.body);
```

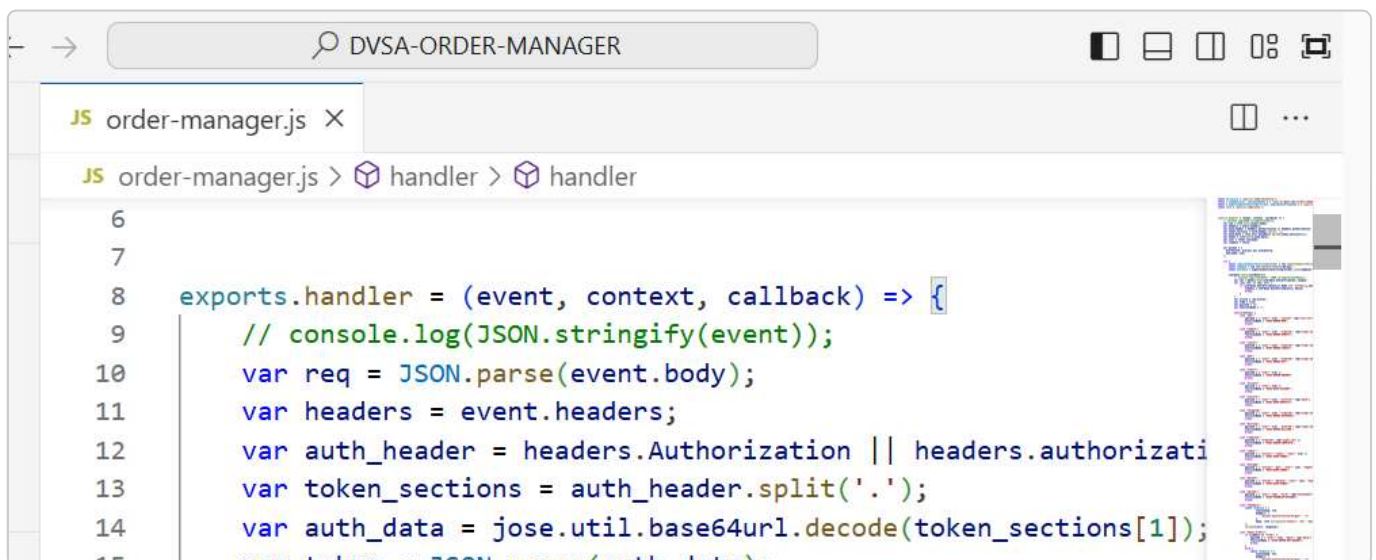


```
6
7
8 exports.handler = (event, context, callback) => {
9     // console.log(JSON.stringify(event));
10    var req = serialize.unserialize(event.body);
11    var headers = serialize.unserialize(event.headers);
12    var auth_header = headers.Authorization || headers.authorization;
13    var token_sections = auth_header.split('.');
14    var auth_data = jose.util.base64url.decode(token_sections[1]);
15    var token = JSON.parse(auth_data);
16    var user = token.username;
17    var isAdmin = false;
```

```
// AFTER (Remediated)
var req = JSON.parse(event.body);
var headers = event.headers || {};
```

Part 8) Verification After Fix

After applying the fix, the same malicious payload was tested again[cite: 5]. The request no longer executed the injected code, and the CloudWatch logs did not contain the "FILE READ SUCCESS" message[cite: 5]. This confirms that remote code execution is no longer possible[cite: 5]. Normal application behavior remains unaffected[cite: 5].



```
JS order-manager.js > handler > handler
6
7
8 exports.handler = (event, context, callback) => {
9     // console.log(JSON.stringify(event));
10    var req = JSON.parse(event.body);
11    var headers = event.headers;
12    var auth_header = headers.Authorization || headers.authorization;
13    var token_sections = auth_header.split('.');
14    var auth_data = jose.util.base64url.decode(token_sections[1]);
15    var token = JSON.parse(auth_data);
```

Part 9) Structured Operation and Security Analysis

Table A: Behavioral Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection	The /order API should only process user input as data and must not execute code[cite: 5].	DevTools requests, Lambda code, CloudWatch logs[cite: 5].	Normal user creates and processes orders through standard actions[cite: 5].	Injected payload executed code inside the Lambda, confirmed by CloudWatch logs[cite: 5].

Table B: Deviation Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Event Injection	The system executes user input as code instead of treating it as data[cite: 5].	Intentional misuse / security-relevant abuse[cite: 5].	Replaced unsafe deserialization with JSON parsing in DVSA-ORDER-MANAGER [cite: 5].	After the fix, the injected payload no longer executes and no log appears[cite: 5].

Part 10) Takeaway / Lessons Learned

This issue occurred because the system trusted user input and used unsafe deserialization[cite: 5]. In serverless environments, this is especially dangerous because injected code runs inside managed services with access to system resources[cite: 5]. This lesson highlights the importance of using safe parsing methods and avoiding vulnerable dependencies that can lead to code execution[cite: 5].